

慶應義塾大学 理工学部 情報工学科 自然言語処理 (2026/06/25)

11 - 言語を生成 (デコーディング) する

情報工学科 准教授 高道 慎之介



Takamichi Laboratory
慶應義塾大学 高道研究室

講義予定

第XX回	日付	内容（順次変わっていくので予想）	
第01回	2026/04/09	イントロダクション	事前知識の確認
第02回	2026/04/16	自然言語処理のための機械学習に入門する	
第03回	2026/04/23	言語学に入門する	言語とは何だろう
第04回	2026/04/30	単語をベクトルで表現する	自然言語処理の基礎技術
第05回	2026/05/07	言語モデルで文章を統計的に扱う	
第06回	2026/05/14	系列にラベリングする	
第07回	2026/05/21	演習 1	これまでの復習
第08回	2026/05/28	Transformer を理解する	大規模言語モデル時代の技術
第09回	2026/06/11	言語モデルを事前学習する	
第10回	2026/06/18	言語モデルを転移学習する	
第11回	2026/06/25	言語を生成 (デコーディング) する	
第12回	2026/07/02	言語やラベルを評価する	おまけ
第13回	2026/07/09	自然言語処理から音声言語処理へ	
第14回	2026/07/16	演習 2	これまでの復習
期末試験	2026/07/XX	(日程は後日アナウンス)	

復習：大規模言語モデル

言語モデル：単語列から次の単語を予測する統計モデル

$$P(w_{N+1} | w_1, w_2, \dots, w_N)$$

次の単語を出力する確率 これまでの文脈 N 単語が与えられている

慶應義塾大学 理工学部 情報工学科 (Department of Information and Computer Science) は、最先端のコンピュータ技術から、それらを社会に役立てるための知能システムまでを幅広くカバーする **???**

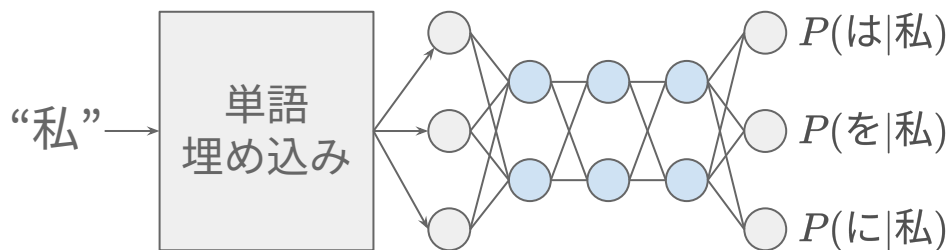
単語予測を繰り返せば文章を生成できる。

- (LLMでは単語ではなくサブワードを扱うが簡単のため単語と表記．扱いは同様)

* 簡単のため「次の」と表記しているが、「前の」や「間の」単語の確率を予測するものも、同様に言語モデルと呼ぶ．次の単語を予測する言語モデルは、自己回帰型言語モデルと呼ばれる。

ニューラル言語モデル

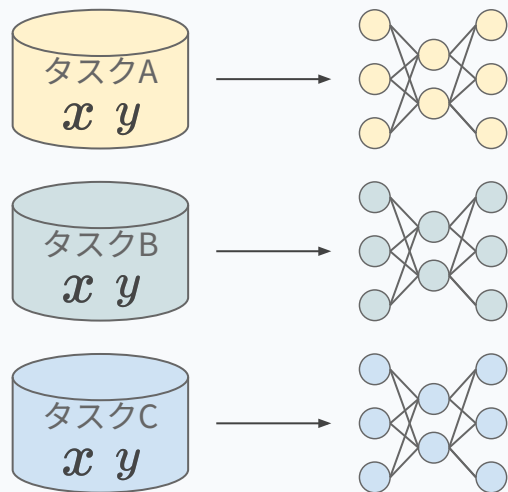
入力 (= 文脈) に対し、出力候補 (= 単語) の確率を出力する。



伝統的な機械学習 vs. 最近の機械学習

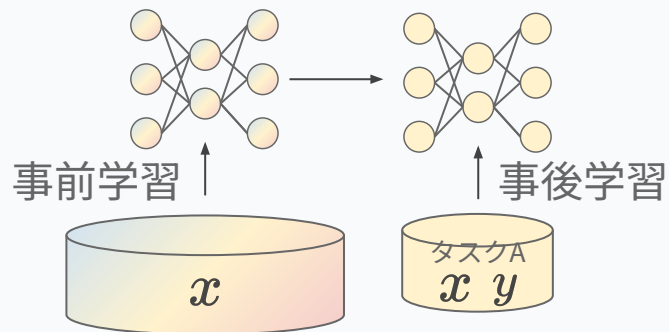
伝統的な機械学習

1. データ (x, y) のペアを大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. (x, y) を使って $f()$, $g()$ を学習
 - a. supervised learning



最近の機械学習

1. x のみ (or y のみ) を大量に用意
2. モデル $f()$, $g()$ を乱数で初期化
3. x (or y) を使って $f()$, $g()$ を学習
4. データ (x, y) のペアを少量だけ用意
5. (x, y) を使って $f()$, $g()$ を微調整
 - a. “supervised fine-tuning (SFT)”



Decoder-only 型の事前学習



次トークン予測 (next token prediction)で学習

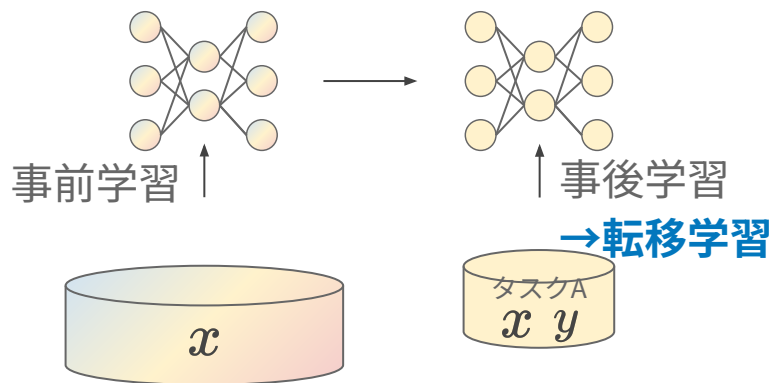
- t 番目の単語を，それ以前の単語から予測

Encoder-decoder 型と比べて

- KV キャッシュによる計算高速化が可能 (課題)
- 推論コストが小さい
 - encoder は self-attention で双方向計算を行うため重い
 - decoder は過去トークンのみ
- 長文に対する推論コストが小さい

転移学習とは

新しいタスクに対する有効な仮説を効率的に見つけ出すために、
一つ以上の別のタスクで学習された知識を保持・適用する問題を指す。



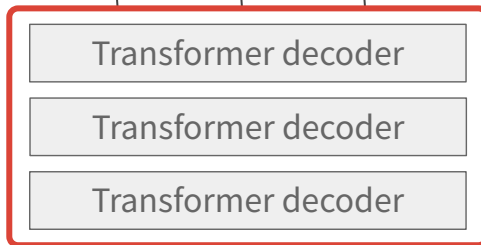
数百～数千例

	トークン数 [兆]
gpt-oss120B	数兆
Llama3.1	15~
DeepSeek-v3	14.8
Qwen3	36
GPT-3	0.5
OLMo 2	3.9

学習済みモデルをタスク特化にする full fine-tuning

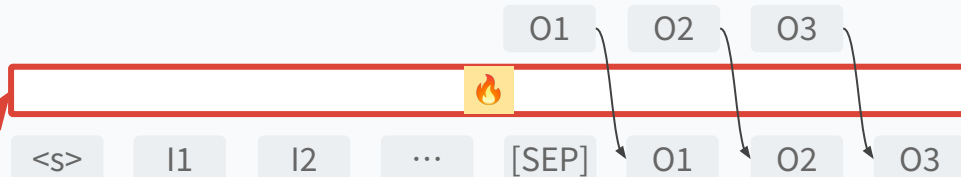
Next token prediction で
事前学習

I eat noodl e ...

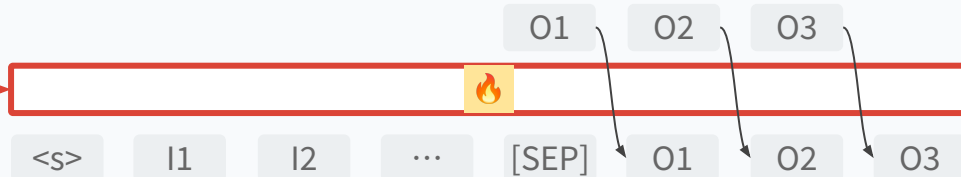


<s> I eat noodl e

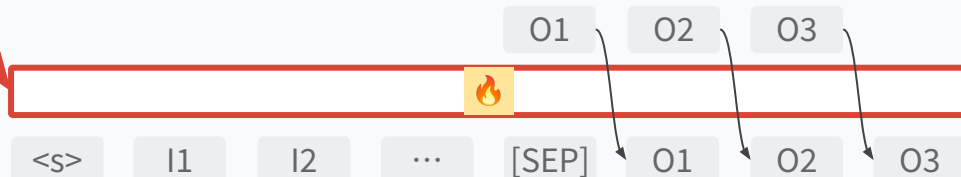
機械翻訳データで学習 (日本語文 I → 英語文 O)



Q&Aデータで学習 (質問文 I → 回答文 O)



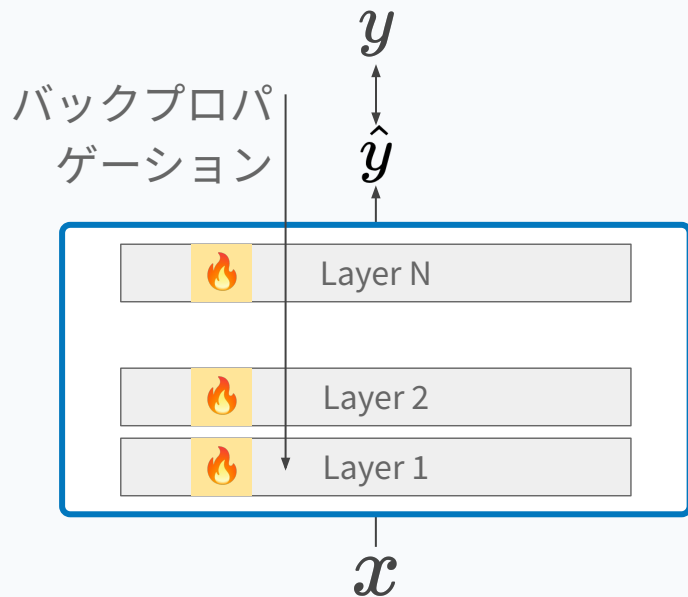
要約データで学習 (元文 I → 要約文 O)



Full fine-tuning vs. Parameter-efficient fine-tuning

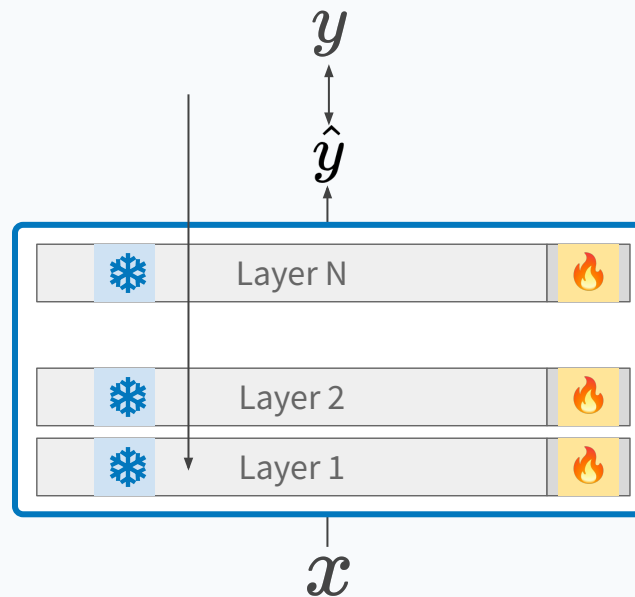
Full fine-tuning

事前学習済みモデルのすべてのパラメータを更新する



Parameter-efficient fine-tuning

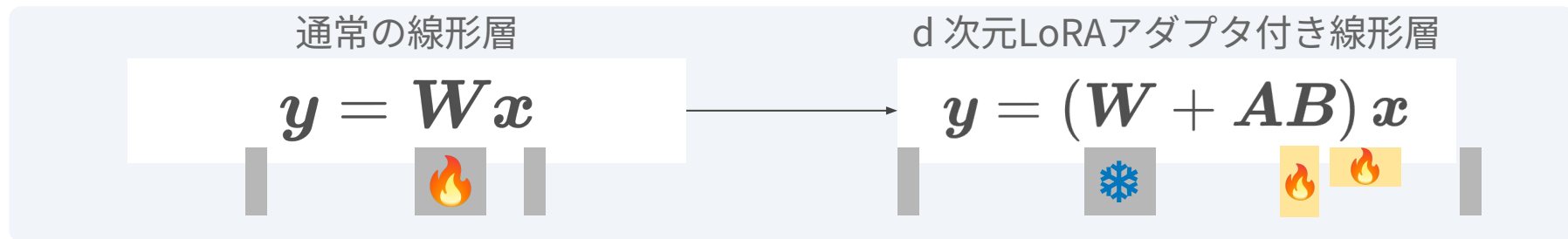
追加したパラメータあるいは元のモデルの一部パラメータのみを更新する



Full fine-tuning vs. Parameter-efficient fine-tuning

要素	Full fine-tuning (Full FT)	Parameter-efficient fine-tuning (PEFT)
モデル	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$
勾配	$N_{\text{param}} \times \text{size}(\text{float16}) \rightarrow 40\text{GB}$	$N_{\text{train}} \times \text{size}(\text{float16}) \rightarrow \text{数十MB}$
最適化器	モデルパラメータ数 (例えば 10^9) $N_{\text{param}} \times \text{size}(\text{float16}) \times 2 \rightarrow 80\text{GB}$	学習するパラメータ数 (例えば 10^5) $N_{\text{train}} \times \text{size}(\text{float16}) \times 2 \rightarrow \text{数十MB}$
ストレージ	元のモデル (N_{param}) + FT済みモデル (N_{param}) $\rightarrow 80\text{GB}$	元のモデル (N_{param}) + 追加モデル (N_{train}) $\rightarrow 40\text{GB} + \alpha$
性能	一般的に良い	FTよりは悪い
必要データ	多くのデータが必要	FTより少なくて良い

Low-Rank Adaptation (LoRA)



学習済みの行列に対し、低ランク行列を追加してその行列のみを更新。

$$W^{(\text{new})} \leftarrow W + AB$$

W は正方行列としているが、
正方行列でなくてもよい。

$\mathbb{R}^{N \times N}$

$\mathbb{R}^{N \times N}$

$\mathbb{R}^{N \times d}$ $\mathbb{R}^{d \times N}$



更新しない

$$\left\{ \begin{array}{l} d=1 \\ (1\text{パターン}) \end{array} \right. \begin{bmatrix} 6 \\ 4 \\ 2 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \end{bmatrix} = \begin{bmatrix} 6 & 3 & 0 \\ 4 & 2 & 0 \\ 2 & 1 & 0 \end{bmatrix}$$
$$\left\{ \begin{array}{l} d=2 \\ (2\text{パターン}) \end{array} \right. \begin{bmatrix} 6 & 7 \\ 4 & 8 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 3 & 7 \\ 4 & 2 & 8 \\ 2 & 1 & 9 \end{bmatrix}$$

今日の目的： 大規模言語モデルから言語を生成する

映画の超かぐや姫みた？

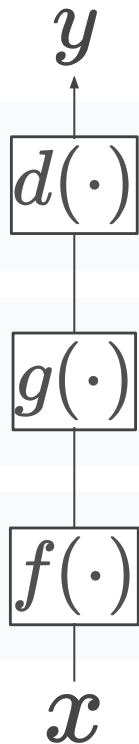
学習済み (decoder-only) LLM

まだ観てないんだよね。どうだった？面白かった？

タイトルのちょっと気になるけど、「かぐや姫の物語」とは別物っぽいよね。もしよかったら、どんな感じの映画か教えてくれたら嬉しい。

基本的な考え方

復習：x から y の予測



決定 (decision) → この部分に相当！

- 出力候補のうちから 1 つを決定すること

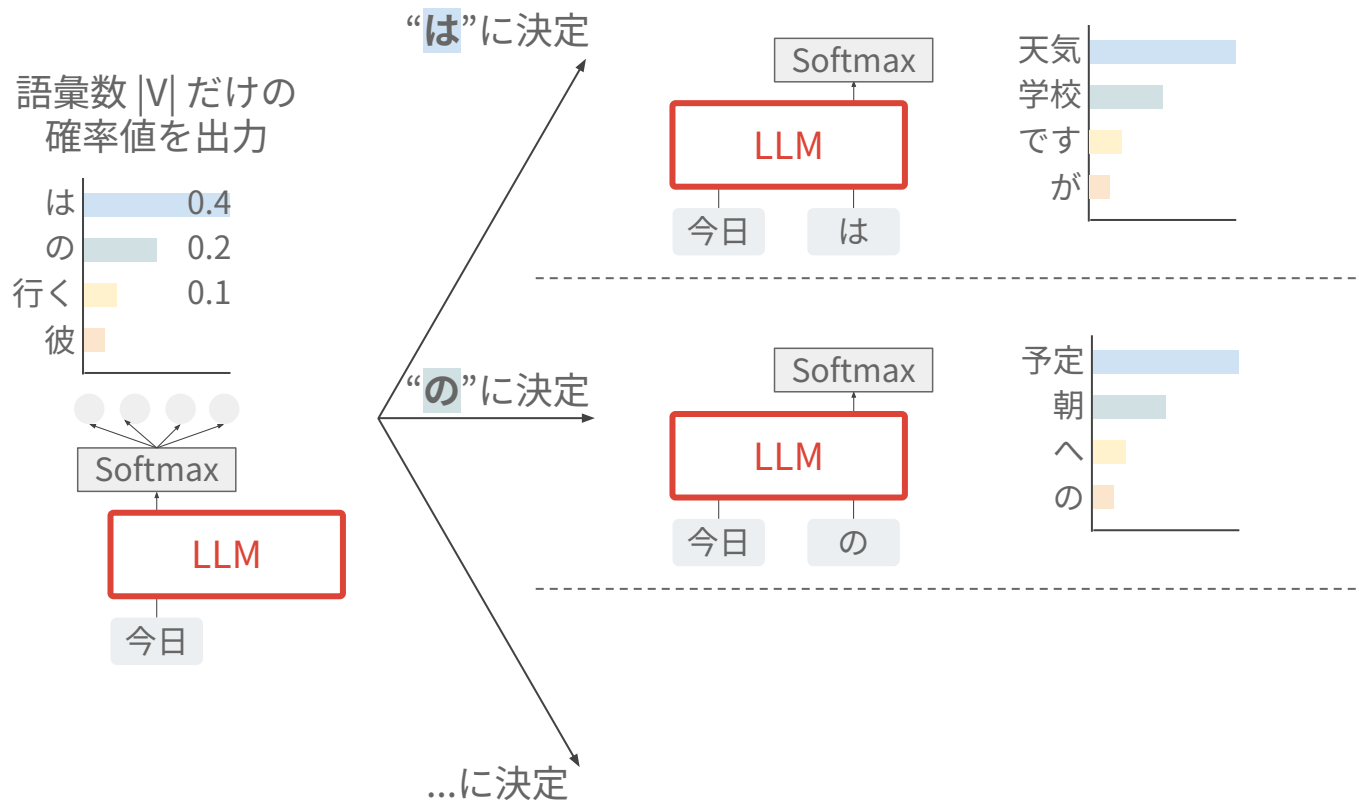
スコア計算 (score calculation)

- 特徴に基づいて、出力候補とそのスコアを計算すること

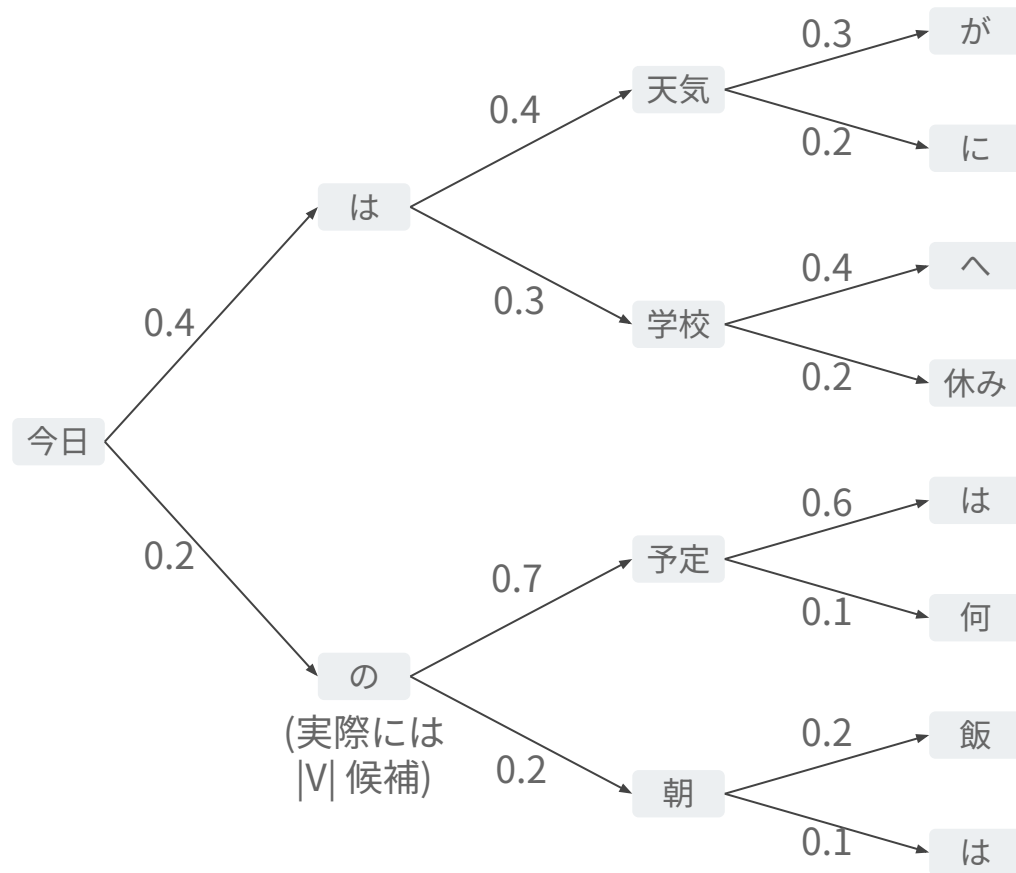
特徴抽出 (feature extraction)

- 意味のある特徴を入力テキストから抽出すること

Decoder-only LLM では、選んだ単語によって 次の単語の確率が変わる



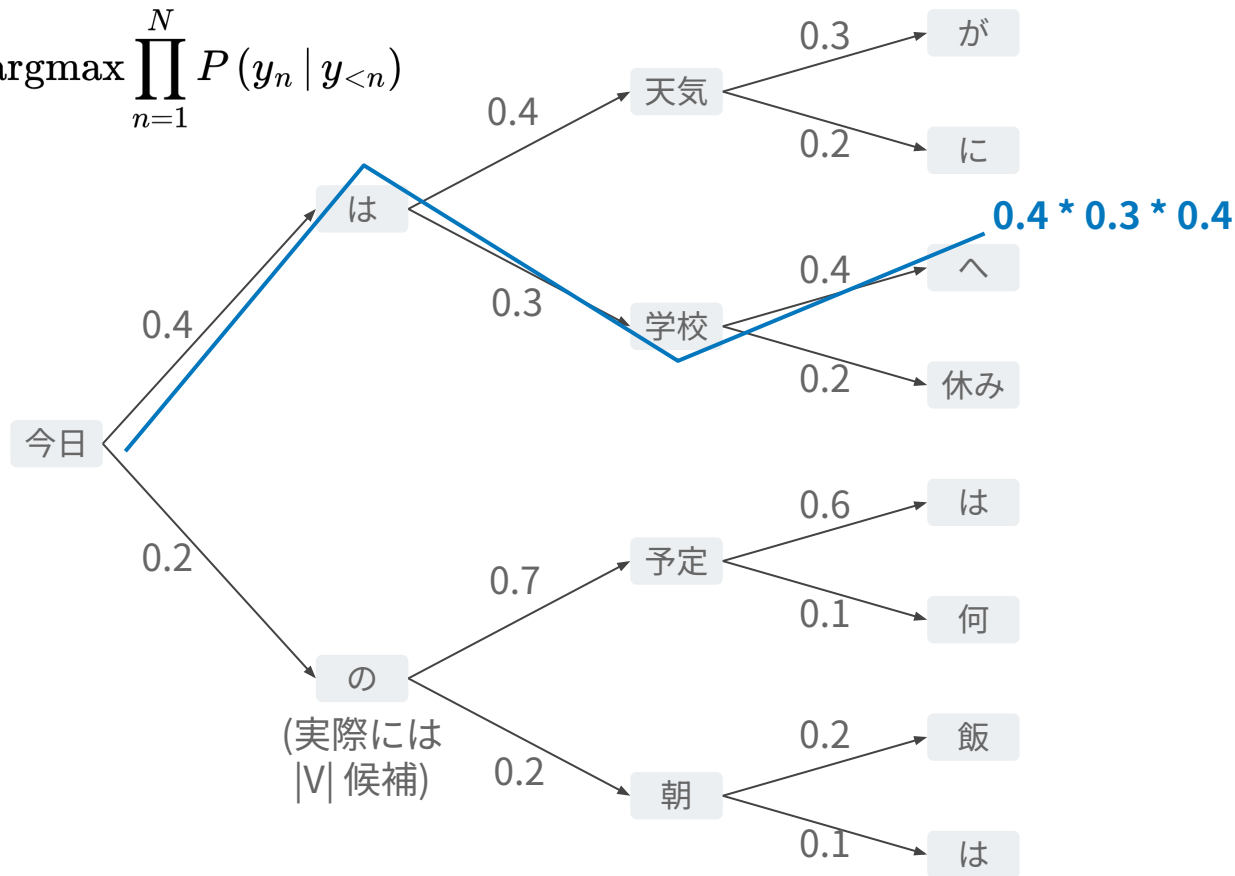
出力の候補はグラフと見做すことができる



生成においてやること：最尤パスを求めること

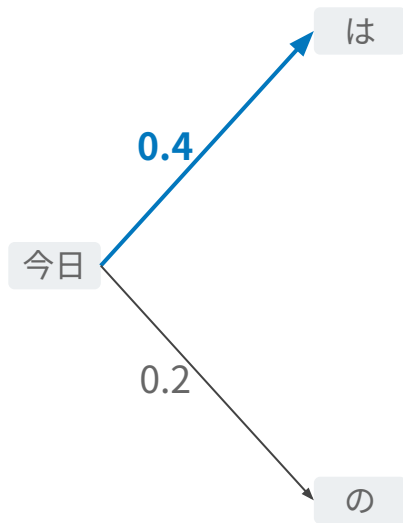
(=確率の積が最も高くなる単語列を求めること)

$$\hat{\mathbf{y}} = \operatorname{argmax} P(\mathbf{y}) = \operatorname{argmax} \prod_{n=1}^N P(y_n | y_{<n})$$

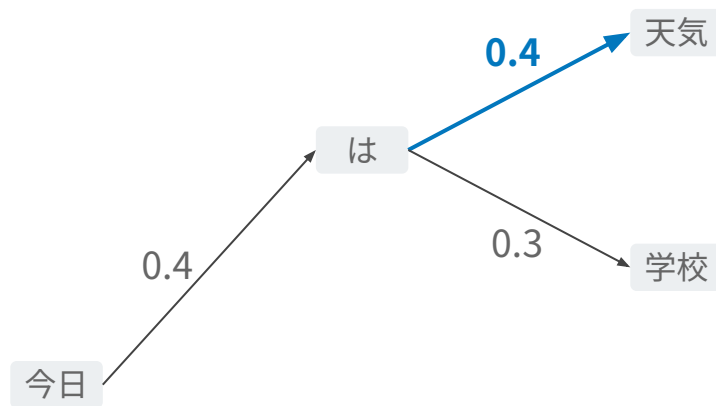


方法① Greedy search

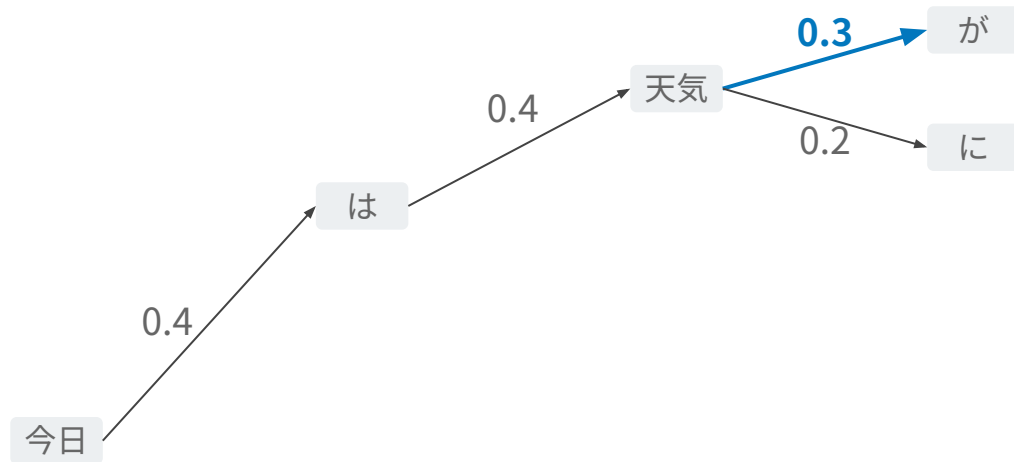
次の単語候補のうち，最も高いものを選ぶ



次の単語候補のうち，最も高いものを選ぶ



次の単語候補のうち、最も高いものを選ぶ



これを繰り返し <EOS> で終了.

プログラミングしてみよう (greedy search)

<https://github.com/takamichi-lab/nlp-lecture-keio/blob/main/2026/codes/11/decoding.ipynb>

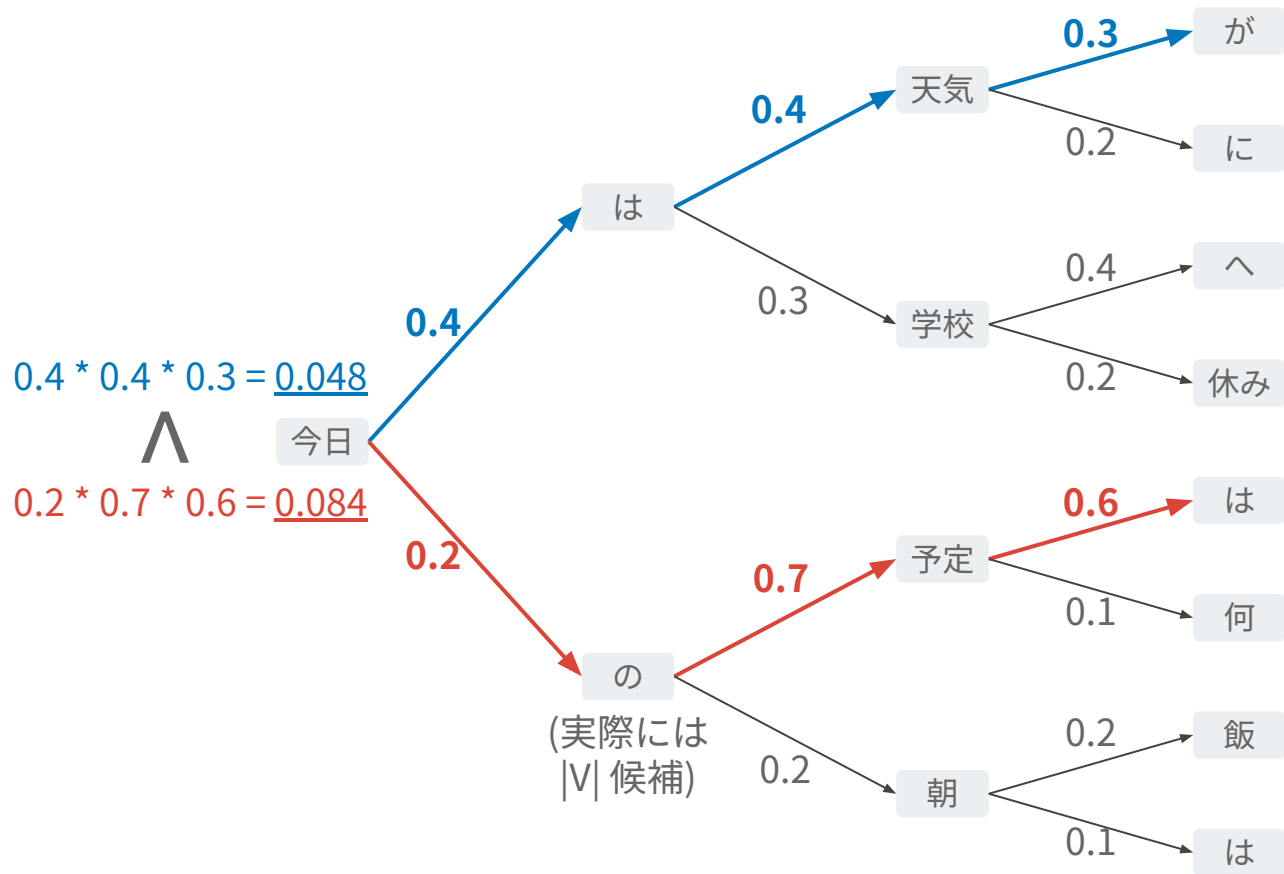
greedy search

```
text = "慶應義塾大学"
tokenized_input = tokenizer.encode(text, add_special_tokens=False, return_tensors="pt").to(model.device)

start_time = time.perf_counter()

with torch.no_grad():
    output = model.generate(
        tokenized_input,
        max_new_tokens=5, # 生成するトークン数
        num_beams=1, # beam searchのビーム幅
        do_sample=False,
        return_dict_in_generate=True,
        output_scores=True,
    )
```

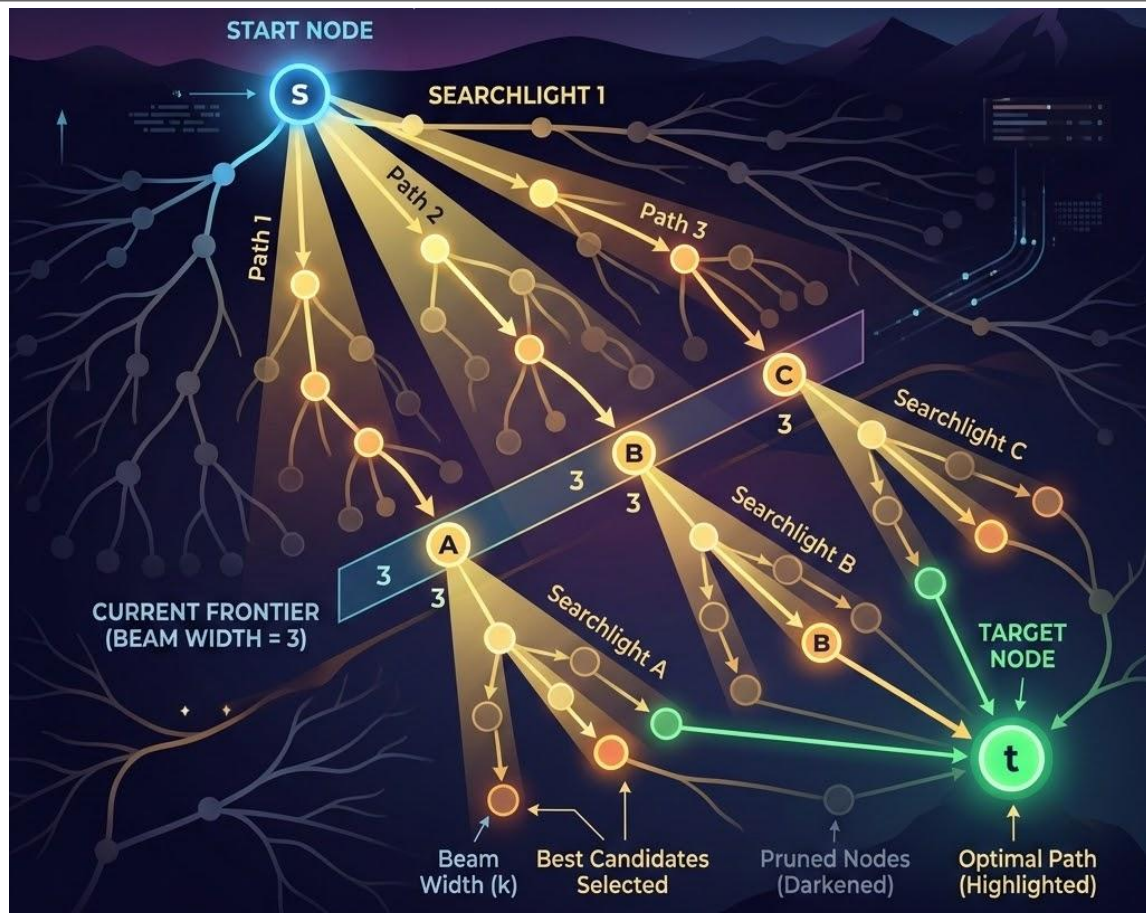
Greedy search の結果が最尤パスとは限らない



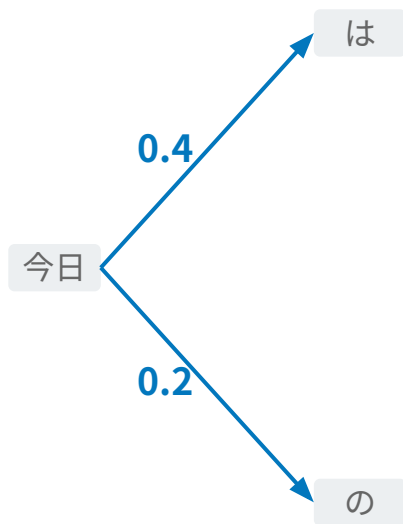
方法②Beam search (幅優先探索の1つ)

最もスコアの高い N 個を残そう

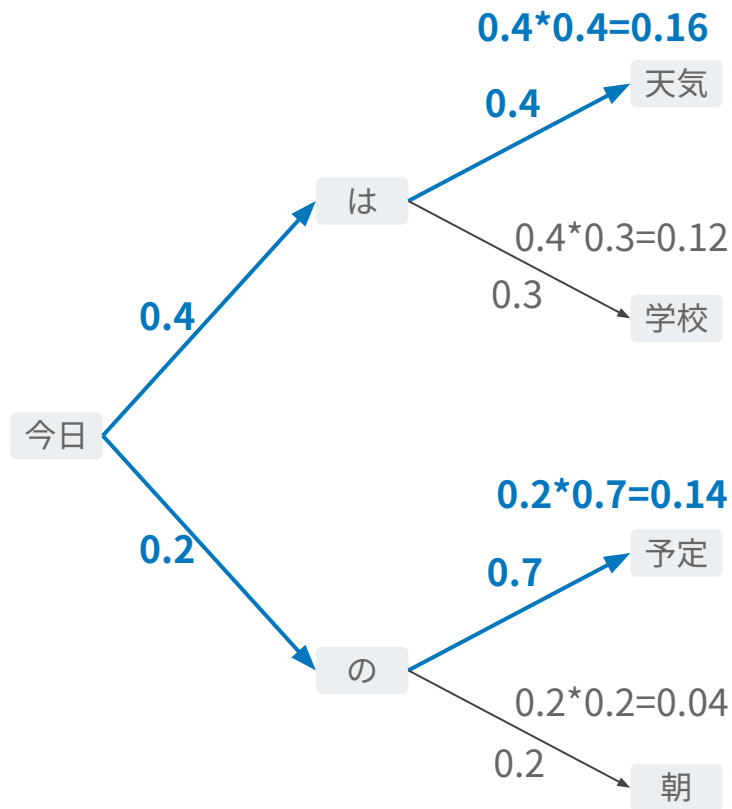
考え方：最もスコアの高い B 個を残そう



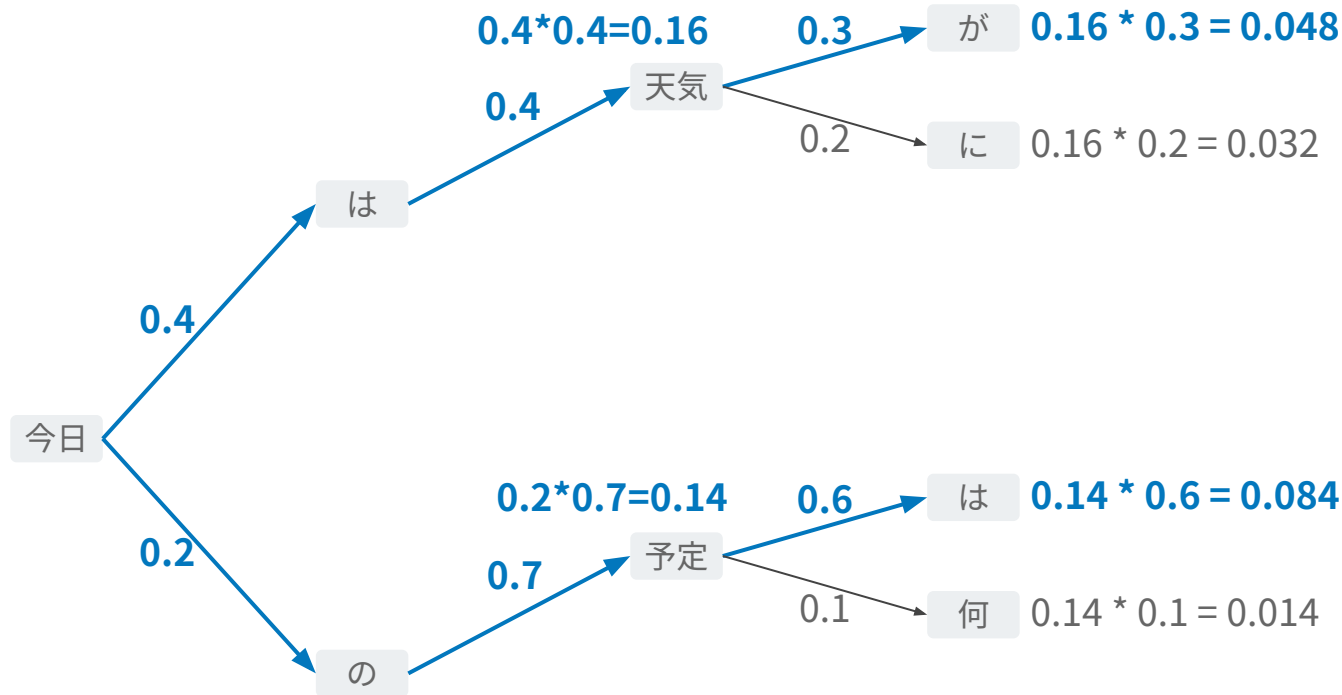
最もスコアの高い B 個を残そう (B=2の例)



最もスコアの高い B 個を残そう (B=2の例)



最もスコアの高い B 個を残そう (B=2の例)



<EOS> までたどり着いたパスのうち、スコアの最も高いものを選択

B の設定について

- $B = 1$: greedy search と等価
- $B = |V|^N$: 全探索 (N は単語列の長さ)
 - 現実的ではない
- $B = 8, 16$ のような小さい値に設定することが一般的であり、推論時のハイパーパラメータになる

プログラミングしてみよう (beam search)

<https://github.com/takamichi-lab/nlp-lecture-keio/blob/main/2026/codes/11/decoding.ipynb>

Beam search

```
text = "慶應義塾大学"
tokenized_input = tokenizer.encode(text, add_special_tokens=False, return_tensors="pt").to(model.device)

start_time = time.perf_counter()

with torch.no_grad():
    output = model.generate(
        tokenized_input,
        max_new_tokens=5, # 生成するトークン数
        num_beams=5, # beam searchのビーム幅
        do_sample=False,
        return_dict_in_generate=True,
        output_scores=True,
    )
```

最尤サンプリングからランダムサンプリングへ

問題 1：最尤推定は繰り返し文を生成しやすい

Next token prediction は繰り返し文に対し高い確率を出しやすいため、繰り返しにスコアペナルティを付与するなどの解決策はある。

llm 同じ文を繰り返す

回答精度が上がる」という発見を報告しています。



G-gen Tech Blog

<https://blog.g-gen.co.jp> : トラブルシューティング :

PaLM 2で同じ文章が何度も繰り返される事象への対処法

2024/02/26 — 当記事では、生成 AI モデルである PaLM 2 のチャットボットを構築した際に、モデルからの回答文で同じ文章が何度も繰り返されてしまう事象の解決策を紹介 ...



Reddit · r/LocalLLaMA

40 件以上のコメント · 2 年前 :

生成中にLLMが繰り返しにハマっちゃう原因って何？

したがって、過度の繰り返しがある場合、モデルは停止する必要があることを認識すべきです。なぜなら、トレーニング文書には同じ文が過度に繰り返される例 ...

PaLM 2 の `chat-bison` や `text-bison` などのモデルを使用して作成したチャットボットに対して、以下のような質問を送信したとします。

- ```
1 # 質問文
2 クラウドエンジニアの採用面接を想定して、1000文字以内の自己PR文を生成してください。
```

PaLM 2 からは以下のような回答が返ってきます。1回の回答で同じ文章が何度も繰り返されており、自己 PR にはとても使用できない文章になっていることがわかります。

- ```
1 # PaLM 2 の回答
2 クラウドエンジニアとして、私はクラウドコンピューティングの深い知識と経験を持ち、クラウド
3
4 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
5
6 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
7
8 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
9
10 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
11
12 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
13
14 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
15
16 私は、クラウドコンピューティングの分野で5年以上の経験を持ち、大手企業のクラウドインフラ
17
```

プログラミングしてみよう (繰り返し文の生成)

<https://github.com/takamichi-lab/nlp-lecture-keio/blob/main/2026/codes/11/decoding.ipynb>

最尤推定は反復を生じさせやすい

```
▶ text = "慶應義塾大学"
  tokenized_input = tokenizer.encode(text, add_special_tokens=False, return_tensors="pt").to(model.device)

  with torch.no_grad():
      output = model.generate(
          tokenized_input,
          max_new_tokens=300, # 生成するトークン数
          num_beams=1, # beam searchのビーム幅
          do_sample=False,
      )

  print(tokenizer.decode(output[0])) # ID を文字列に変換して出力
```

問題 2 : 短い文が選ばれやすい

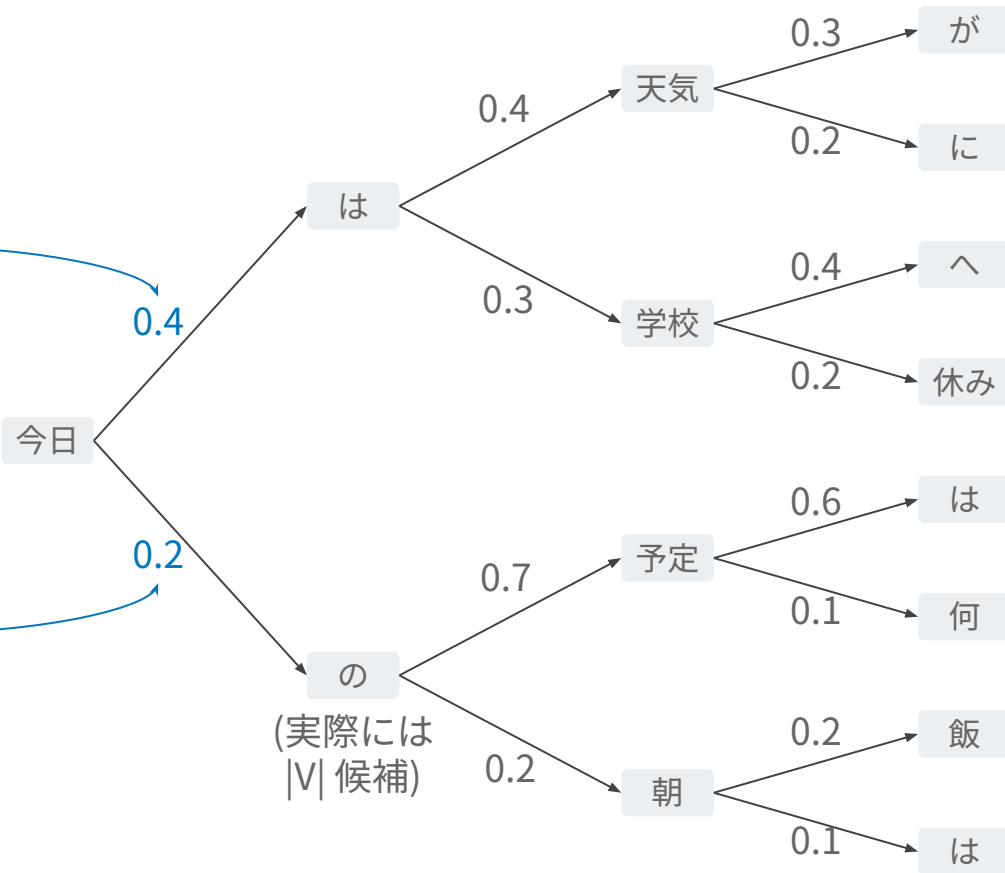
スコアは確率 (常に1以下) の積なので, 系列長が長いほどスコアは小さくなる.
短い系列長にペナルティを入れるなどの解決策はある.



解決策：最尤からサンプリングへ

「確率の高い候補を残す」ではなく、
確率に基づいてランダムに選ぶ。

ただし、確率値の顕著に低いものは
不自然な文になるため、ある程度
確率値が高いものだけを使う！

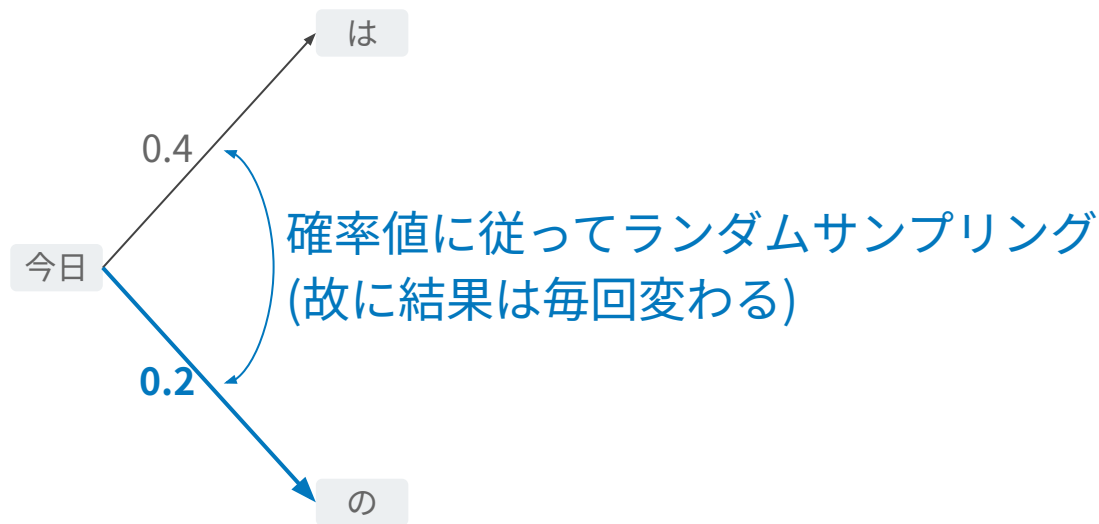


ランダムサンプリングへ

(現代の LLM API に標準搭載されている)

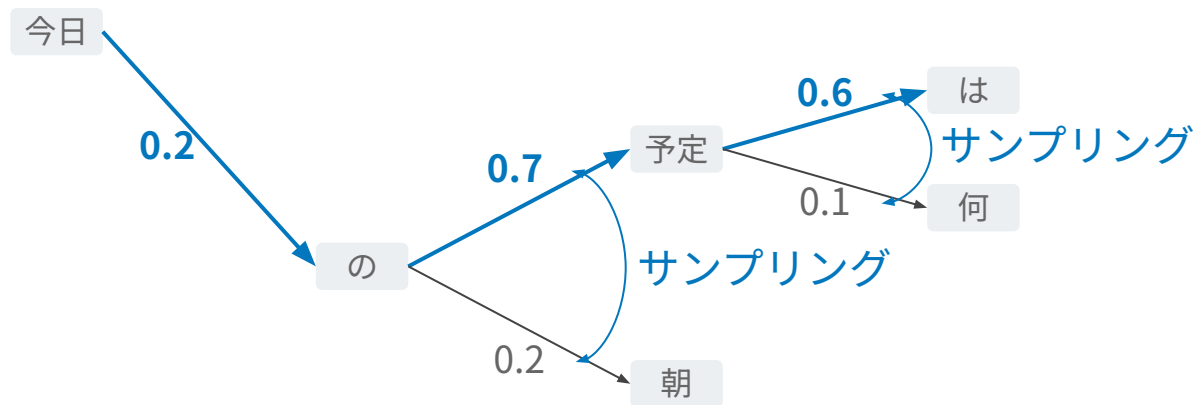
方法①Ancestral sampling

単語毎にサンプリングする (greedy search のサンプリング版?)



方法①Ancestral sampling

単語毎にサンプリングする (greedy search のサンプリング版?)

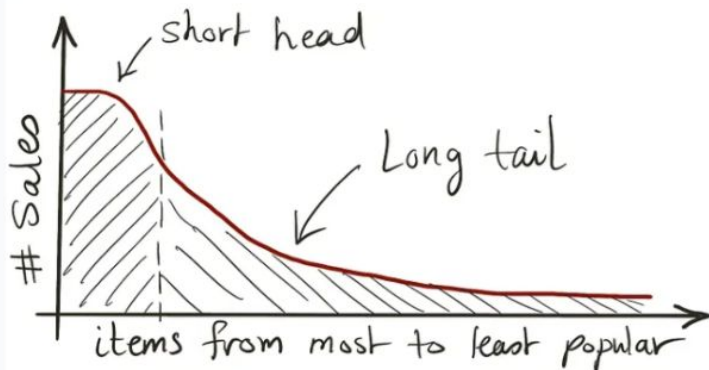


Ancestral sampling の何が悪い？ → long tail 問題

ロングテール分布

超大規模データでも稀発単語は多数存在し、分布は**ロングテール**となる。

n-gram (単語列) ではこの傾向がより顕著に現れる。



<https://medium.com/codex/machine-learning-the-long-tail-paradox-1cc647d4ba4b>

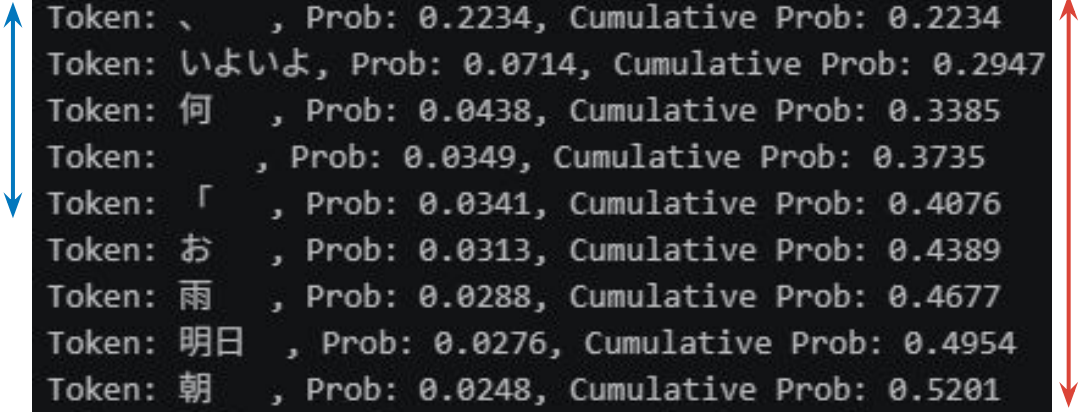
- next token prediction における次単語の確率は long tail になる
- 確率値の低いものは不自然な文になりやすいものが多い
- Long tail に含まれる各単語の確率値は小さいがその総計は大きい
→ **不自然な文を出力する確率が高い**
- **Short head だけを選んでサンプリングすればよいのでは？**

プログラミングしてみよう (ancestral sampling)

<https://github.com/takamichi-lab/nlp-lecture-keio/blob/main/2026/codes/11/decoding.ipynb>

方法②Top- k sampling & Top- p sampling

top- k : 確率値の高い
上位 k 個だけを残す
(例えば $k = 5$)



Token: 、	Prob: 0.2234	Cumulative Prob: 0.2234
Token: いよいよ	Prob: 0.0714	Cumulative Prob: 0.2947
Token: 何	Prob: 0.0438	Cumulative Prob: 0.3385
Token:	Prob: 0.0349	Cumulative Prob: 0.3735
Token: 「	Prob: 0.0341	Cumulative Prob: 0.4076
Token: お	Prob: 0.0313	Cumulative Prob: 0.4389
Token: 雨	Prob: 0.0288	Cumulative Prob: 0.4677
Token: 明日	Prob: 0.0276	Cumulative Prob: 0.4954
Token: 朝	Prob: 0.0248	Cumulative Prob: 0.5201
Token: また	Prob: 0.0215	Cumulative Prob: 0.5417

top- p : 累積確率値が
 p の要素だけを残す
(例えば $p = 0.5$)

方法③Temperature sampling

候補を打ち切る方法以外に、確率分布を鈍らせたり尖らせたりする方法がある。

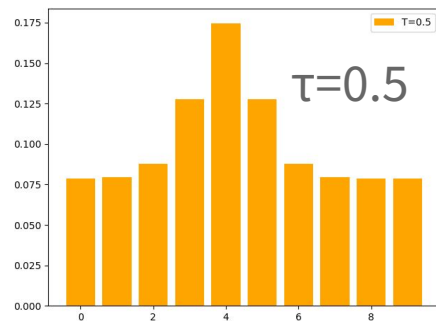
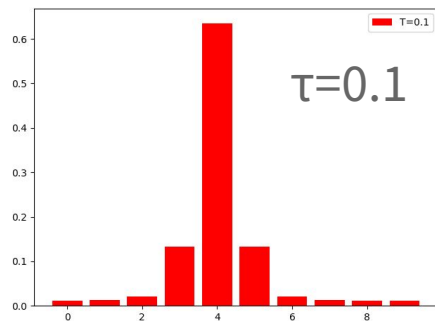
通常の softmax 関数

$$\text{softmax}(\mathbf{y})_j = \frac{\exp(y_j)}{\sum_{\text{all } k} \exp(y_k)}$$

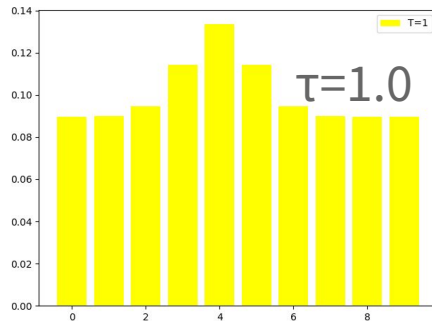
温度付き通常の softmax 関数

$$\text{softmax}(\mathbf{y}, \tau)_j = \frac{\exp(y_j/\tau)}{\sum_{\text{all } k} \exp(y_k/\tau)}$$

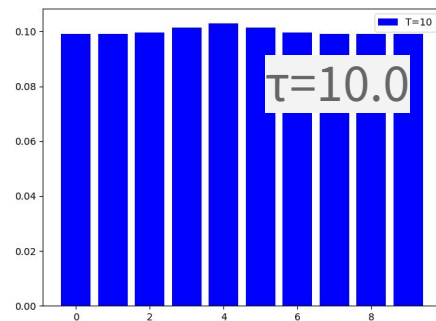
温度が低いほど
確率ピークを強調



通常のsoftmax
関数の出力と同じ



温度が高いほど
一様分布に近づく



プログラミングしてみよう (top-k, top-p, temperature)

<https://github.com/takamichi-lab/nlp-lecture-keio/blob/main/2026/codes/11/decoding.ipynb>

Top-k sampling, Top-p sampling



The image shows a Jupyter Notebook interface. At the top right, there is a toolbar with icons for up, down, edit, delete, and a menu. The main area contains a code cell with the following Python code:

```
# text に続く単語の確率分布を計算, 確率の高い順に表示
text = "明日は"
tokenized_input = tokenizer.encode(text, add_special_tokens=False, return_tensors="pt").to(model.device)

with torch.no_grad():
    output = model.generate(
        tokenized_input,
        max_new_tokens=1, # 生成するトークン数
        do_sample=True,
        return_dict_in_generate=True,
        output_scores=True,
    )
# 表示
next_token_logits = output.scores[0]
```

まとめ

Conclusion

まとめ

- 最尤パス推定
 - Greedy search: 単語ごとに最尤のものを選ぶ
 - Beam search: あらかじめ決めた候補数を残して探索
- サンプリング
 - Ancestral sampling: 単語ごとにサンプリング
 - Top-k sampling: 確率の高い k 単語を残してサンプリング
 - Top-p sampling: 累積確率が p 以下の単語を残してサンプリング
 - Temperature sampling: 確率分布を{尖らせて, 鈍らせて}サンプリング

課題 (GPU推奨)

Huggingface にある事前学習済みLLMを読み込め。

- https://huggingface.co/models?pipeline_tag=text-generation&num_parameters=min:0,max:9B&library=transformers&language=ja,en&sort=trending

The screenshot shows the Hugging Face website's 'Models' page. The left sidebar contains navigation tabs for 'Main', 'Tasks', 'Libraries', 'Languages', 'Licenses', and 'Other'. Under 'Tasks', 'Text Generation' is selected. Below this, there are filters for 'Parameters' (ranging from <1B to >500B) and 'Libraries' (including PyTorch, TensorFlow, JAX, Transformers, etc.). The main content area displays a list of models under the heading 'Models 1,202'. The models are sorted by 'Trending'. The first few models shown are:

- Jackrong/Qwopus3.6-27B-Coder-MTP-GGUF (Text Generation, 0.5B, Updated about 12 hours ago, 245k downloads, 290 likes)
- LiquidAI/LFM2.5-8B-A1B (Text Generation, 8B, Updated 15 days ago, 199k downloads, 638 likes)
- LiquidAI/LFM2.5-1.2B-Instruct (Text Generation, 1B, Updated 9 days ago, 120k downloads, 617 likes)
- unsloth/LFM2.5-8B-A1B-GGUF (Text Generation, 8B, Updated 26 days ago, 67.6k downloads, 73 likes)
- llm-jp/llm-jp-4-8b-thinking-gguf (Text Generation, 9B, Updated 3 days ago, 111 downloads, 5 likes)
- LiquidAI/LFM2.5-350M (Text Generation, 0.4B, Updated 21 days ago, 66k downloads, 340 likes)
- LiquidAI/LFM2.5-1.2B-Thinking (Text Generation, 1B, Updated Mar 30, 49.5k downloads, 367 likes)
- LiquidAI/LFM2.5-350M-Base (Text Generation, 0.4B, Updated Mar 31, 36.4k downloads, 20 likes)
- aifeifei798/DarkIdol-Llama-3.1-8B-Instruct-1.2-Uncensored (Text Generation, 8B, Updated Jul 30, 2024, 1.43k downloads, 233 likes)
- CohereLabs/aya-expense-8b (Text Generation, 8B, Updated Jan 10, 22.2k downloads, 438 likes)

課題 (GPU推奨)

複数のモデルに複数の推論アルゴリズムを使用し，傾向を調査せよ．例えば：

- Beam search の効果はモデルサイズに影響するか？
- Top-k, top-p アルゴリズムはどういうモデルを使うときに有効か？
- ハイパーパラメータを変えたときにどのような挙動になるか？
 - beam search 幅， k ， p ， τ
- 他に思いついたこと

何か 1 つを選び調査せよ．